

SQL TEMEL KOMUTLAR EL KİTABI

Teknik Referans ve Uygulama Kılavuzu

1 SQL Dilinin Taksonomisi

SQL (Structured Query Language), ilişkisel cebir tabanlı bir dildir. Komutlar, veri tabanı üzerindeki etki alanlarına göre dört ana kategoriye ayrılır:

- **DDL (Data Definition Language):** Veri yapısını (şema) tanımlar.
- **DML (Data Manipulation Language):** Veri üzerinde kayıt düzeyinde işlem yapar.
- **DQL (Data Query Language):** Veriyi sorgular ve projeksiyonlar oluşturur.
- **DCL/TCL (Control/Transaction):** Yetkilendirme ve işlem kararlılığını (ACID) yönetir.

2 DDL: Yapısal Tanımlama

Veri tabanı nesnelerinin (tablo, indeks, view) "metrik yapısını" belirleyen komutlardır.

Tablo 1: Temel DDL Komutları

Komut	Açıklama	Örnek
CREATE	Yeni nesne oluşturur.	CREATE TABLE Personel (...);
ALTER	Yapıyı günceller.	ALTER TABLE Personel ADD Maas...;
DROP	Nesneyi kalıcı olarak siler.	DROP TABLE Personel;
TRUNCATE	Veriyi boşaltır (Tablo kalır).	TRUNCATE TABLE Personel;

3 DML: Veri İşleme (Satır Yönetimi)

Tablo içindeki "tuple" (satır) vektörleri üzerinde yapılan operasyonlardır.

INSERT: Yeni bir veri vektörü ekler.

```
1 INSERT INTO Kitaplar (ISBN, Baslik) VALUES ('978-605', 'Veri Tabanı Sistemleri');
```

UPDATE: Mevcut verinin durumunu (state) değiştirir.

```
1 UPDATE Kitaplar SET Stok = Stok - 1 WHERE ISBN = '978-605';
```

4 DQL: Veri Sorgulama ve İlişkisel Cebir

Sorgulama, bir R ilişkisinden istenen özniteliklerin projeksiyonunu alma işlemidir. Matematiksel olarak:

$$\sigma_{koul}(\pi_{stunlar}(Tablo))$$

```

1 SELECT Ad, Soyad           -- Projeksiyon
2 FROM Ogrenciler           -- Kaynak
3 WHERE Gano > 3.0           -- Secim (Filter)
4 ORDER BY Soyad ASC;       -- Siralama

```

5 İlişkisel Operatörler: JOIN

İlişkisel modelin gücü, tabloların anahtarlar ($PK - FK$) üzerinden birbirine bağlanmasından gelir.

- **INNER JOIN:** Her iki tabloda kesişen kayıtları getirir ($A \cap B$).
- **LEFT JOIN:** Soldaki tablonun tamamını ve sağdaki eşleşenleri getirir.

6 Veri Bütünlüğü Kısıtları (Constraints)

Veri tabanının kararlı halini koruyan "sınır şartları" şunlardır:

1. **PRIMARY KEY:** Satırı uzayda tekilleştirir ($NOT NULL + UNIQUE$).
2. **FOREIGN KEY:** Tablolar arası referansal bağımlılığı kurar.
3. **CHECK:** Değerin fonksiyonel bir aralıkta olmasını sağlar.

7 ER Modelinden İlişkisel Modele Dönüşüm

ER diyagramındaki kavramlar SQL tablolarına sistematik olarak dönüştürülür.

Dönüşüm Kuralları

1. Her **Varlık (Entity)** → Bir tablo.
2. Her **Basit Öznelik** → Bir sütun.
3. **Anahtar Öznelik** → PRIMARY KEY.
4. **1:N İlişki** → N tarafına FOREIGN KEY eklenir.
5. **M:N İlişki** → Ara tablo oluşturulur.
6. **Çok Değerli Öznelik** → Ayrı tablo.

Örnek

ER: Öğrenci (OgrNo, Ad) Ders (DersKod, DersAd) Öğrenci –(Kayıt)– Ders (M:N)
İlişkisel karşılığı:

```

1 CREATE TABLE Ogrenci (
2     OgrNo INT PRIMARY KEY,
3     Ad NVARCHAR(50)
4 );
5
6 CREATE TABLE Ders (
7     DersKod VARCHAR(10) PRIMARY KEY,
8     DersAd NVARCHAR(50)
9 );
10
11 CREATE TABLE Kayit (
12     OgrNo INT,
13     DersKod VARCHAR(10),
14     PRIMARY KEY(OgrNo, DersKod),
15     FOREIGN KEY (OgrNo) REFERENCES Ogrenci(OgrNo),
16     FOREIGN KEY (DersKod) REFERENCES Ders(DersKod)
17 );

```

8 Normalizasyon

Normalizasyon, veri tekrarını azaltmak ve anomalileri engellemek için yapılan matematiksel ayrıştırma işlemidir.

8.1 1NF (Birinci Normal Form)

Koşul:

- Her hücre atomik olmalıdır.

- Tekrarlı grup bulunmamalıdır.

Yanlış yapı:

| OgrNo | Ad | Telefonlar | |——|——|———| | 101 | Hasan | 555, 544 |

Doğru yapı:

Ogrenci(OgrNo, Ad)

Telefon(OgrNo, Telefon)

8.2 2NF

Koşul:

- 1NF sağlanmalı.
- Bileşik anahtara kısmi bağımlılık olmamalı.

Örnek: PRIMARY KEY (OgrNo, DersKod)

Eğer "Ad" sadece OgrNo'ya bağlıysa → ayrı tabloya alınır.

8.3 3NF

Koşul:

- 2NF sağlanmalı.
- Transitif bağımlılık olmamalı.

Yanlış:

Personel(PersonelNo, DepartmanNo, DepartmanAdi)

DepartmanAdi → DepartmanNo'ya bağlıdır.

Doğru:

Departman(DepartmanNo, DepartmanAdi)

Personel(PersonelNo, DepartmanNo)

9 Aggregate Fonksiyonlar ve GROUP BY

SQL'de toplulaştırma fonksiyonları:

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()

```
1 SELECT DersKod, AVG(Puan) AS Ortalama
2 FROM Notlar
3 GROUP BY DersKod;
```

HAVING Kullanımı

```
1 SELECT DersKod, AVG(Puan) AS Ortalama
2 FROM Notlar
3 GROUP BY DersKod
4 HAVING AVG(Puan) > 70;
```

10 JOIN Türleri

10.1 INNER JOIN

```
1 SELECT O.Ad, D.DersAd
2 FROM Kayit K
3 INNER JOIN Ogrenci O ON K.OgrNo = O.OgrNo
4 INNER JOIN Ders D ON K.DersKod = D.DersKod;
```

10.2 LEFT JOIN

Soldaki tablodaki tüm kayıtlar gelir.

```
1 SELECT O.Ad, D.DersAd
2 FROM Ogrenci O
3 LEFT JOIN Kayit K ON O.OgrNo = K.OgrNo
4 LEFT JOIN Ders D ON K.DersKod = D.DersKod;
```

11 Alt Sorgular (Subqueries)

Alt sorgu, başka bir sorgunun içinde çalışan sorgudur.

Scalar Subquery

```
1 SELECT Ad
2 FROM Ogrenci
3 WHERE Gano > (SELECT AVG(Gano) FROM Ogrenci);
```

IN Operatörü

```
1 SELECT Ad
2 FROM Ogrenci
3 WHERE OgrNo IN (SELECT OgrNo FROM Kayit);
```

12 Transaction Yönetimi ve ACID

Bir işlem (transaction), ya tamamen gerçekleşir ya da hiç gerçekleşmez.

ACID Özellikleri

- **Atomicity** – Bölünemezlik
- **Consistency** – Tutarlılık
- **Isolation** – İzolasyon
- **Durability** – Kalıcılık

```
1 BEGIN TRANSACTION;
2
3 UPDATE Hesap
4 SET Bakiye = Bakiye - 1000
5 WHERE HesapNo = 1;
6
7 UPDATE Hesap
8 SET Bakiye = Bakiye + 1000
9 WHERE HesapNo = 2;
10
11 COMMIT;
12 -- veya
13 ROLLBACK;
```

13 View (Görünüm)

View, sanal tablodur.

```
1 CREATE VIEW BasariliOgrenciler AS
2 SELECT Ad, Soyad, Gano
3 FROM Ogrenci
4 WHERE Gano > 3.0;
```

14 Index (İndeks)

İndeks, arama hızını artırır.

```
1 CREATE INDEX idx_ad  
2 ON Ogrenci (Ad);
```

Not:

- Okuma hızlanır.
- Yazma işlemi biraz yavaşlar.

15 Mini Proje Önerisi

Öğrencilerden aşağıdaki sistemi tasarlamaları beklenir:

- En az 4 tablo
- En az 1 M:N ilişki
- PRIMARY KEY ve FOREIGN KEY tanımları
- 5 adet SELECT sorgusu
- 1 adet GROUP BY sorgusu
- 1 adet Subquery

Örnek konular:

- E-Ticaret Sistemi
- Hastane Otomasyonu
- Kütüphane Yönetimi
- Öğrenci Bilgi Sistemi